MapReduce, and the problem *Spark was built to fix*

We have a dataset that doesn't fit on one machine, spread in Parquet across a cluster, and we want to compute something over all of it. Say the average `release_speed` by `pitch_type` across ten seasons. The data lives on ten nodes. The honest question is: what is the dumbest correct way to do this, and why does the dumbest correct way turn out to be most of the answer?

The dumbest correct way was published by Google in 2004 and it is called **MapReduce**. We are not going to use MapReduce in this course. Spark replaced it for good engineering reasons. But Spark is, at its core, a much faster and more pleasant MapReduce, and you cannot understand why Spark is shaped the way it is without first understanding the thing it improved on. So we start here, with the ancestor.

The insight: move the code, not the data

Remember Dean's numbers from Unit 1. Reading a megabyte sequentially from disk is cheap; sending a megabyte across the network is an order of magnitude more expensive. Now look at our problem again. The Statcast data is already sitting on ten DataNodes. We could pull all ten nodes' worth of data to one machine and compute the average there, but that means shipping the entire dataset across the network to a single bottleneck. Sounds slow, because it is.

The MapReduce insight is to turn the problem inside out. Instead of moving terabytes of data to where the code is, move the kilobytes of code to where the data is. Each node runs your computation on its own local partition, reading from its own local disk, and only the small summarized results travel across the network. This is **data locality**, and it is the single idea that makes cluster computing economical. We will see Spark fight to preserve locality for the rest of the unit.

The shape: map, shuffle, reduce

MapReduce expresses every computation as two functions you write and one phase the framework runs for you.

The **map** function runs on each input record independently and emits zero or more key-value pairs. For our average-velocity-by-pitch-type job, the map function takes one pitch and emits `(pitch_type, release_speed)`. It does this for every pitch on its local partition, in parallel across all ten nodes, touching only local disk. No coordination, no network.

The **reduce** function takes a key and all the values that were emitted for that key, from anywhere in the cluster, and combines them into a result. For our job, reduce takes `("FF", [95.1, 94.8, 96.0, ...])` and produces the average. There is one logical reducer per distinct key.

Between map and reduce sits the phase you don't write and the phase that costs you everything: the **shuffle**. The map outputs are scattered across all ten nodes, but reduce needs all the values for a given key gathered in one place. So the framework partitions every map output by key, sends each pair to the node responsible for that key, and groups them. Every `("FF", ...)` pair in the cluster, no matter which node produced it, has to physically travel to the one node that will reduce the `"FF"` key.

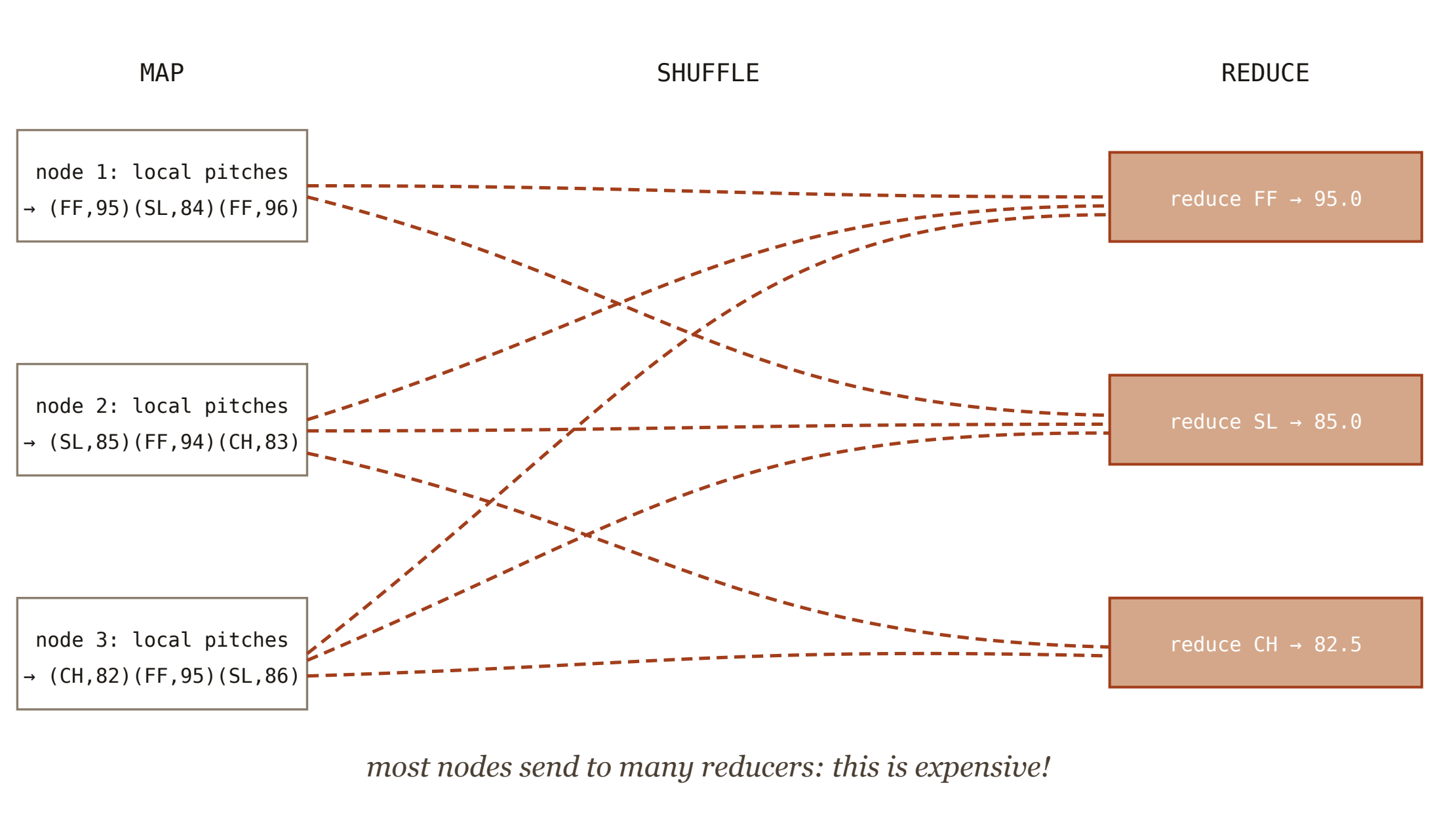

FIG 2.1 *The MapReduce dataflow. Map runs locally on each node with no coordination. Reduce runs once per key. The shuffle in the middle is an all-to-all data movement: a pair produced on any node may need to reach a reducer on any other node. The map and reduce phases parallelize beautifully; the shuffle is where the network bill comes due.*

This is worth sitting with, because the shape never goes away. Map is embarrassingly parallel and local. Reduce is parallel across keys. The shuffle is an all-to-all data movement across the network, and it is the expensive part. When we get to Spark shuffles, we are going to be looking at this exact picture with different labels. The framework changed; the physics did not.

Why MapReduce was a breakthrough, and where it hurt

MapReduce was a genuine breakthrough, and it is worth naming why. Before it, writing a program that ran reliably across a thousand machines was a research project. You had to handle the machine that died mid-job, the task that ran slow because its disk was failing, the network hiccup that dropped a result. MapReduce did all of that for you. If a node died during the map phase, the framework simply re-ran that node's map tasks somewhere else, because map tasks are deterministic functions of their input and can be recomputed from scratch. Failure recovery, which Unit 1 told you is the central problem at scale, came essentially for free. You wrote two functions and the framework handled the cluster.

So why don't we use it? Two reasons, and both of them shaped Spark directly.

The first is that MapReduce writes to disk between every step. The output of a reduce phase lands in HDFS, and if your computation needs another round of map-and-reduce (and real computations always do), the next round reads that output back from HDFS, processes it, and writes again. Every intermediate result is a full read-and-write to replicated, on-disk storage. For a single map-reduce pass this is fine. But consider an iterative algorithm, say the gradient descent you saw in Applied Linear Algebra, which repeats the same computation dozens or hundreds of times. Each iteration pays the full disk-and-replication tax on its intermediate state. The algorithm spends almost all of its time reading and writing HDFS, not computing.

The second reason is that the programming model is too low-level. Expressing "average `release_speed` by `pitch_type`, but only for pitches faster than 90, joined against a table of pitcher names" as a chain of raw map and reduce functions is painful and error-prone. You find yourself hand-writing the same patterns over and over, and you have no way to ask the framework to optimize across steps, because the framework cannot see inside your map function. It is opaque code, not a declarative description of intent.

What Spark kept and what it changed

Spark kept the good idea and fixed both wounds. It kept data locality, kept the map-shuffle-reduce shape underneath, and kept automatic failure recovery. What it changed was where intermediate results live and how you describe the computation.

For intermediate results, Spark keeps them in memory across steps wherever it can, spilling to disk only when memory runs out. Our 30-iteration job now reads from HDFS once at the start, does all 30 passes in memory, and writes once at the end. The orders-of-magnitude speedups Spark advertised in its early days were almost entirely this one change: stop round-tripping to disk between every step.

For the programming model, Spark lets you describe the whole computation as a chain of high-level operations, then looks at the entire chain before running any of it and figures out an efficient plan. That requires two ideas we are going to spend more time on: a way to record the chain of operations without running it, and a way to recover from failure without writing every step to replicated disk. Both come from a single concept, the lineage of a computation, and it is the heart of how Spark works.

Lineage, laziness, and *the DAG*

We ended last time with two problems Spark had to solve: how to recover from failure without writing every step to replicated disk, and how to optimize across a whole computation instead of one opaque step at a time. Spark solves both with one idea. The idea is that Spark does not store your data and then transform it. Spark stores a *recipe* for your data, and only cooks when you make it.

The RDD as a recipe

The foundational abstraction in Spark is the **RDD**, the Resilient Distributed Dataset. The name is a mouthful, so take it apart. *Distributed*: it represents data partitioned across the cluster, the same partitions we built by hand in Unit 1. *Dataset*: it represents a collection of records. *Resilient*: it can rebuild any lost partition without anyone having saved a copy. That last word is the clever one, and it is where we are headed.

The thing to unlearn coming from pandas is that an RDD is not the data. When you write a line of Spark that "filters the Statcast pitches to those over 90 mph," Spark does not filter anything. It records a note to itself: *this dataset is defined as: take that other dataset, and apply this filter*. The note points back at the dataset it was built from, which points back at the dataset *it* was built from, all the way back to the original files on disk. The data does not exist yet. Only the recipe exists.

Think of a recipe card that says "the sauce is: reduce the stock from the previous card, then whisk in the butter from the card before that." The card doesn't contain sauce. It contains instructions and a pointer to the cards it depends on. You can hold a whole stack of these cards, describing an elaborate dish, and not a single pot is on the stove. This chain of cards, each pointing at the ones it was built from, is the **lineage** of the computation. It is the most important word in this lecture.

Transformations build the recipe; actions cook

Spark operations come in two flavors. A **transformation** takes a dataset and describes a new one: `filter`, `select`, `groupBy`, `join`, `map`. Transformations are lazy. They do no work. They add a card to the stack and return immediately, no matter how much data the dataset "contains," because it contains nothing yet.

An **action** asks for an actual answer that has to leave the cluster: `count`, `collect`, `write`, `show`, `take`. An action cannot be lazy, because you are asking for a number or a file or a screenful of rows, and Spark has to produce it. The moment you call an action, Spark looks at the entire stack of recipe cards leading up to it, plans how to execute them, and only then moves any data.

```
pitches = spark.read.parquet("s3a://sd411/statcast/")   # transformation: nothing runs
fast    = pitches.filter(pitches.release_speed > 90)     # transformation: nothing runs
by_type = fast.groupBy("pitch_type").avg("release_speed") # transformation: nothing runs
by_type.show()                                           # ACTION: now everything runs
```

The first three lines return instantly even on ten seasons of data, because they move nothing. They build a three-card recipe. The fourth line is where the cluster wakes up. This catches every student at least once: you run a notebook cell that "loads and transforms a billion rows" in 40 milliseconds, conclude Spark is magic, then run a `count` and wait two minutes. Spark is not magic. The first cell did nothing. The `count` did everything.

Laziness is advantageous because Spark sees the whole recipe before it runs anything, it can optimize across steps. It can notice that you only ever use three of the eighty columns and never read the other seventy-seven from disk. It can push your `release_speed > 90` filter down into the Parquet reader so the row groups that can't match are skipped (predicate pushdown, from Unit 1). An eager system that ran each line as it saw it could not do any of this. Laziness is what makes the optimizer possible.

Resilience: rebuild rather than replicate

Now the payoff for the recipe model, and the answer to the failure problem MapReduce solved with disk. Suppose Spark is halfway through our job and a node holding one partition of `fast` dies. In MapReduce, you'd recover by re-reading a replicated copy from HDFS. Spark does something cheaper. It still has the lineage: it knows that the lost partition of `fast` is defined as "the over-90 filter applied to partition such-and-such of `pitches`." It also knows where that input partition lives. So it simply re-runs the filter on that one input partition, on some surviving node, and reconstructs exactly the lost partition. No replica was ever stored. The recipe *is* the backup.

This is what the R in RDD buys you. Resilience comes from being able to recompute any partition from its lineage, rather than from having saved a copy of it. It costs nothing during normal operation, because a recipe card is a few bytes of metadata, and it costs only the recomputation of the affected partitions during a failure. For a long iterative job this is dramatically cheaper than MapReduce's write-everything-to-disk approach.

There is a catch, and it is worth naming now because it will return in the future. If the lineage is very long (hundreds of iterations), recomputing a lost partition from the original files means replaying hundreds of steps, which can be slower than the failure was worth. The escape hatch is **checkpointing**: occasionally Spark can write a partition's actual contents to reliable storage and truncate the lineage there, so recovery doesn't have to go all the way back. Caching, which we will cover later in this unit, is the in-memory cousin of this. For now, just hold the idea that lineage has a length, and length has a cost.

The lineage is a DAG

Stack up enough recipe cards and what you have is a graph: each dataset is a node, and an edge runs from each dataset to the datasets it was built from. The graph has a direction (one way for lineage, the other for execution) and it never loops back on itself (a dataset is never built from itself), so it is a **directed acyclic graph**, a DAG. When you call an action, Spark hands this DAG to a component called the DAG scheduler, whose job is to turn the graph into actual units of work.

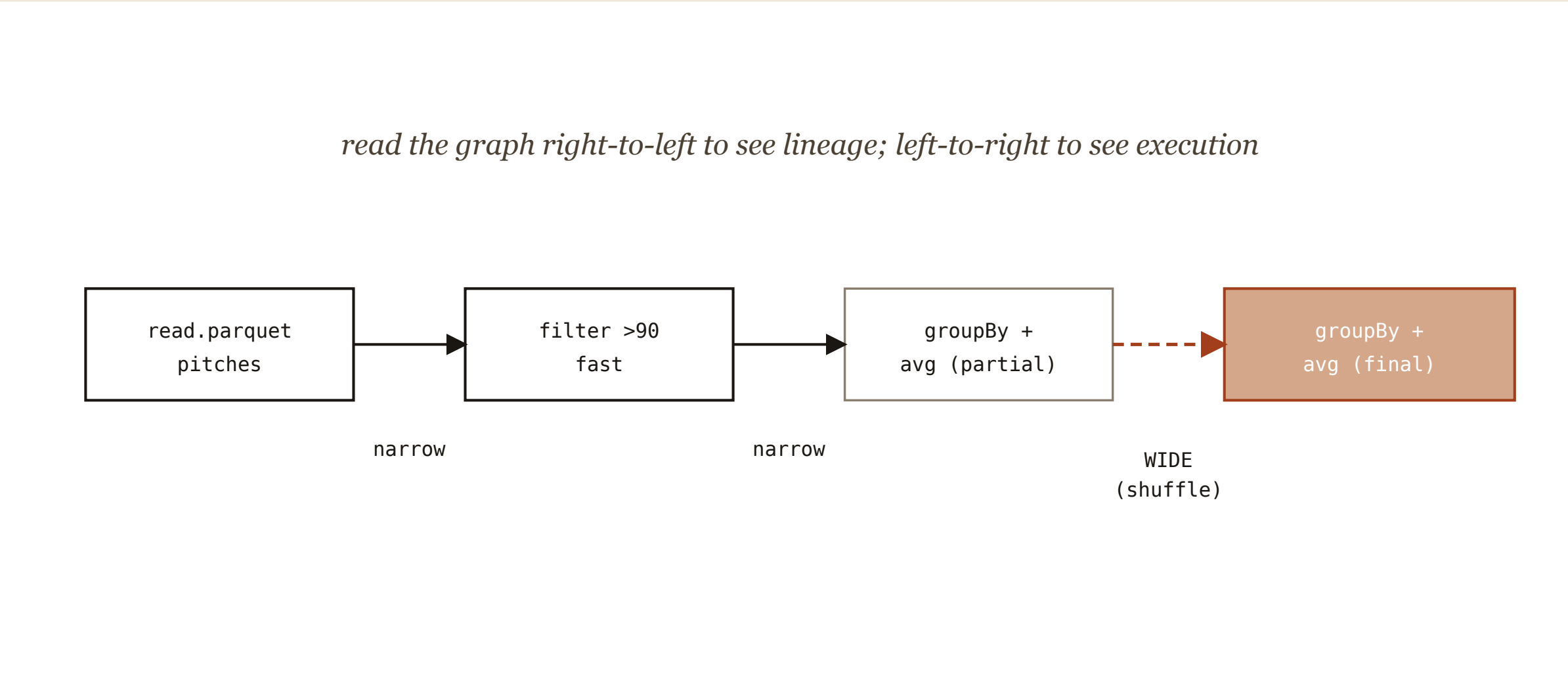

FIG 2.2 *The lineage DAG for our four-line job. Three of the edges are narrow: each output partition depends on one input partition, so the work stays local. One edge is wide: to compute the final average per pitch type, partial results for the same key from every partition must be brought together. That wide edge is a shuffle, and it is the only expensive edge in the graph.*

Narrow and wide dependencies

Look closely at the edges, because the difference between two kinds of edge is the difference between cheap and expensive, and we will use this distinction for the rest of the unit. An edge is a **narrow dependency** if each partition of the output depends on at most one partition of the input. `filter` is narrow: to filter partition 3, you need only partition 3. `map` and `select` are narrow. Narrow operations require no data movement; each node works on its own partition and produces a corresponding output partition, entirely locally.

An edge is a **wide dependency** if a partition of the output depends on many or all partitions of the input. `groupBy` is wide: to compute the average for `"FF"`, you need every `"FF"` row, and those are scattered across all partitions. `join` is usually wide. A wide dependency cannot be satisfied locally. It requires the all-to-all data movement we met in MapReduce. It requires a shuffle.

So here is the rule you carry into every Spark job you will ever read: **narrow is free, wide is a shuffle, and shuffles are expensive.** When you look at a DAG, your eye should jump immediately to the wide edges, because those are the only places real cost lives. Everything else is a local pass over data the node already has.

Jobs, stages, and tasks

The DAG scheduler turns the graph into a three-level hierarchy, and you need all three words because the Spark UI is organized around them.

A **job** is everything triggered by one action. Our `show()` is one job. Two actions, two jobs, even if they share upstream work.

A job is cut into **stages** at the wide edges. A stage is a run of narrow operations that can all be pipelined together on each partition without moving data. The moment the DAG hits a wide dependency, the current stage ends, because the next operation needs data from other partitions and therefore cannot start until the shuffle has moved that data. Our job has two stages: read-and-filter-and-partial-aggregate (all narrow, pipelined into one stage), then the shuffle boundary, then final-aggregate. The stage boundaries are exactly the shuffle boundaries. Always. If you can find the shuffles, you can find the stages, and vice versa.

A stage is executed as a set of **tasks**, one task per partition. If the input to a stage has 200 partitions, that stage runs as 200 tasks. A task is the actual unit of work handed to an executor: "run this stage's narrow pipeline on this one partition." Tasks within a stage are independent and run in parallel, as many at a time as you have executor cores. This is the level at which Spark actually does work, and it is the level at which performance problems become visible, because one slow task can hold up an entire stage.

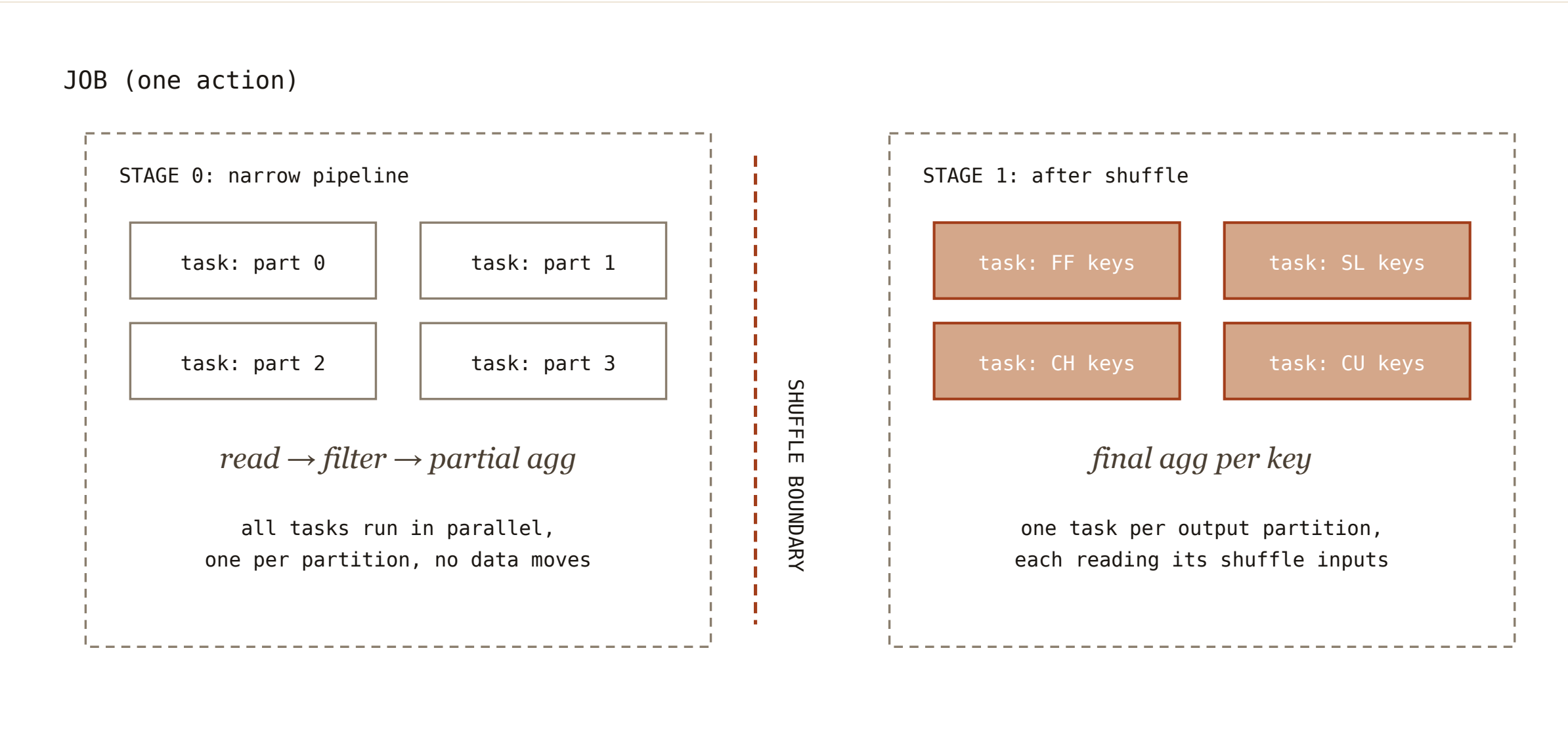

FIG 2.3 *One job, cut into two stages at the shuffle boundary, each stage run as a set of parallel tasks. Stages are bounded by shuffles by definition. Tasks are bounded by partitions: a stage with 200 input partitions runs as 200 tasks. This three-word vocabulary, job, stage, task, is exactly how the Spark UI is laid out, which is why we drilled it here before we ever open the UI.*

That is the whole execution model. You write transformations, which build a lineage DAG. You call an action, which becomes a job. The job is cut into stages at the shuffles. Each stage runs as one task per partition. Every performance topic for the rest of the unit, partitioning, shuffles, joins, skew, and caching, is a consequence of this picture. Before any of that, though, we have been writing RDD-style code in our examples, and almost nobody writes RDD code anymore...

RDDs, DataFrames, and *Spark SQL*

We have been writing code that filters and groups RDDs, and we said almost nobody writes RDD code anymore. That sounds like a contradiction, so lets resolve it. The RDD is the foundation; everything in Spark still runs as RDDs underneath. But you rarely write at that level, for the same reason you rarely write assembly: there is a higher-level way to say what you want that is both easier to write and faster to run. Understanding why it is faster is the point of this lecture, and it sets up the optimizer next time.

The problem with the RDD API

An RDD holds objects, and Spark knows nothing about what is inside them. When you write `pitches.filter(lambda p: p.release_speed > 90)` on an RDD, you hand Spark a Python function. To Spark, that function is a black box. It cannot see that you only touch the `release_speed` field. It cannot see that `release_speed` is a number. It cannot reorder your filter relative to a later operation, because it has no idea what your function does. It can only do exactly what you said, in exactly the order you said it, on opaque objects it cannot inspect.

There is a second, quieter cost. Those objects are Java or Python objects on the heap, with all the memory overhead that implies: object headers, pointers, boxed numbers, garbage-collection pressure. A single `release_speed` value that is logically eight bytes of double might cost you forty or more bytes once it is a boxed object in a collection. At billions of rows, that overhead is not a footnote. It is the difference between fitting in memory and spilling to disk.

The DataFrame: structure Spark can see

A **DataFrame** is a distributed table with a known schema: named columns, each with a type. This is the same mental object you already have from pandas and from SQL, how convenient! The difference from an RDD is not the API surface. The difference is that Spark now knows the structure. It knows there is a column called `release_speed` and that it holds doubles. It knows you have eighty columns and your query touches three.

Because Spark can see the structure, it can do all the things the opaque RDD denied it. When you write `df.filter(df.release_speed > 90).select("pitch_type", "release_speed")`, you are not handing Spark a black-box function. You are handing it a description of intent that it can read, rearrange, and rewrite. It can prune the seventy-seven columns you never mention. It can push the filter into the Parquet reader. It can decide the order of operations for itself. None of this was possible when the operation was a Python lambda.

The RDD gives you arbitrary, fully general code: you can do anything you can express in a function. The DataFrame gives up that generality in exchange for legibility: you describe *what* you want in terms Spark understands, and in return Spark gets to decide *how*. For the analytic workloads in this course, that is a trade you almost always want to make, because Spark optimizes better than you will by hand.

Tungsten: getting out of the JVM's way

Knowing the schema buys a second win, on memory. Because Spark knows every column's type, it does not have to store rows as JVM objects at all. The **Tungsten** execution engine stores DataFrame rows in a compact binary format in off-heap memory: a known layout of raw bytes, no object headers, no pointers, no boxing. That `release_speed` double is eight bytes, sitting where Spark put it, not forty bytes of boxed object the garbage collector has to chase. Spark also generates specialized bytecode at runtime that operates directly on this binary layout, a technique called whole-stage code generation, which collapses a whole chain of narrow operations into a single tight loop over the bytes. We will see its fingerprint in the physical plan next lecture.

The practical upshot: a DataFrame computation over the same data uses far less memory than the equivalent RDD computation and runs the inner loop much faster, before we even count the optimizer's contribution. The structure is not just nicer to write. It is the thing that lets Spark store the data densely and run over it fast.

Spark SQL is the same engine with a different front door

Here is the part that surprises people. Spark SQL, where you write literal SQL strings, is not a separate system bolted onto Spark. It is the same engine as the DataFrame API, reached through a different syntax. These two snippets compile to the identical plan and run identically:

```
# DataFrame API
df.filter(df.release_speed > 90) \
  .groupBy("pitch_type") \
  .avg("release_speed")

# Spark SQL
spark.sql("""
  SELECT pitch_type, AVG(release_speed)
  FROM pitches
  WHERE release_speed > 90
  GROUP BY pitch_type
""")
```

They are two spellings of one intent. Spark parses each into the same internal representation and runs the same optimizer over it. There is no performance reason to prefer one over the other. Choose SQL when the logic reads naturally as a query and you want analysts to be able to read it; choose the DataFrame API when you are composing transformations programmatically or want the type-checking of method calls. They meet in the same place a layer down.

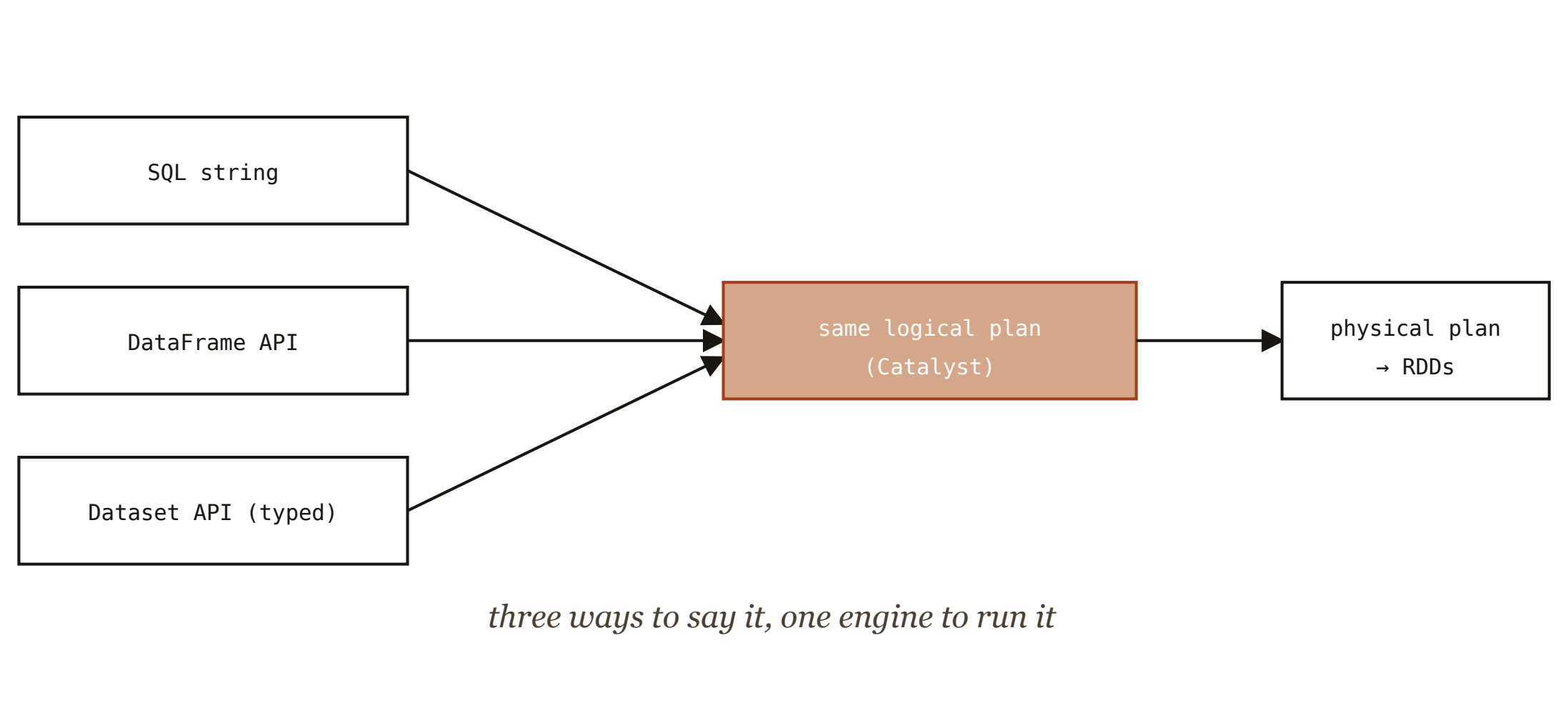

FIG 2.4 *The three structured front doors, SQL, the DataFrame API, and the typed Dataset API, all lower into the same internal logical plan and are optimized by the same component (Catalyst, next lecture). Under that, the optimized plan becomes a physical plan that runs as RDDs. The API you choose is an ergonomics decision, not a performance one.*

So when is each one appropriate?

The honest default for this course is: use DataFrames and Spark SQL for essentially everything. They are faster, lighter on memory, and they let the optimizer help you. The cases where you reach below them to raw RDDs are specific and rare.

Use...	When	Why
DataFrame / SQL	Any structured, analytic query: filters, aggregations, joins, windows over tabular data	Catalyst optimizes it; Tungsten stores it densely; this is 95% of real work
Dataset (typed)	You want compile-time type safety on rows and are in Scala/Java	Same engine, with type checking; less relevant from Python
RDD	Truly unstructured data, or an algorithm the structured API can't express, or you need fine control over partitioning the high-level API hides	Maximum generality, at the cost of the optimizer and Tungsten

A useful way to hold it: the RDD is the assembly language of Spark, and you should know it exists and how it works, because everything compiles down to it and the failure-recovery story lives there. But you write in the high-level language, and you only drop to assembly when the high-level language genuinely cannot say what you need. For Statcast analytics, that almost never happens.

We have now said three times that the DataFrame is faster because Spark can read your intent and rewrite it. That rewriting has a name, and it is the most important machinery in Spark for an operator to understand, because reading its output is how you predict a job's behavior before you run it. That machinery is Catalyst, and it is next.

Catalyst, and reading *a physical plan*

We have said the DataFrame is faster because Spark reads your intent and rewrites it into an efficient plan. The component that does the rewriting is the **Catalyst** optimizer, and this lecture has a blunt goal: by the end of it you should be able to call `.explain()` on a query, read the tree it prints, and predict where the job will spend its time. This is the single most useful skill in the unit. A student who can read a physical plan can debug a Spark job they have never seen. A student who cannot is reduced to guessing.

The pipeline from query to execution

Catalyst takes your query through four stages, and naming them tells you what kind of mistake gets caught where.

First, **parsing** turns your SQL string or DataFrame calls into an unresolved logical plan: a tree of operations where the column and table names are still just names, not yet checked against anything. Syntax errors die here.

Second, **analysis** resolves those names against the actual schemas. It confirms `release_speed` is a real column of type double, that `pitches` is a real table, that the types line up. A misspelled column name dies here, with the `cannot resolve` error you have already met. The output is a resolved logical plan: a correct description of *what* to compute, with no opinion yet on *how*.

Third, **logical optimization** rewrites that plan into an equivalent but cheaper one using a library of rules. This is where the wins from the last lecture actually happen. Three rules matter most for us:

- **Predicate pushdown.** Move filters as close to the data source as possible, ideally into the Parquet reader itself, so row groups that cannot match are never read. This is the Unit 1 idea, now applied automatically.
- **Column pruning.** If the query only ever uses three of the eighty columns, read only those three from disk. On columnar Parquet this is nearly free to exploit and enormously valuable.
- **Constant folding and simplification.** Evaluate constant expressions once, collapse trivially true or false conditions, combine adjacent filters. Small, but free.

Fourth, **physical planning** turns the optimized logical plan into one or more physical plans, each a concrete strategy, and picks one using a cost model. This is where a join becomes specifically a broadcast join or a sort-merge join (more to follow), and where the abstract "aggregate" becomes a specific hash-aggregate operator. The chosen physical plan is then handed to whole-stage code generation, which fuses pipelined operators into tight generated loops, and finally executed as the RDD stages and tasks.

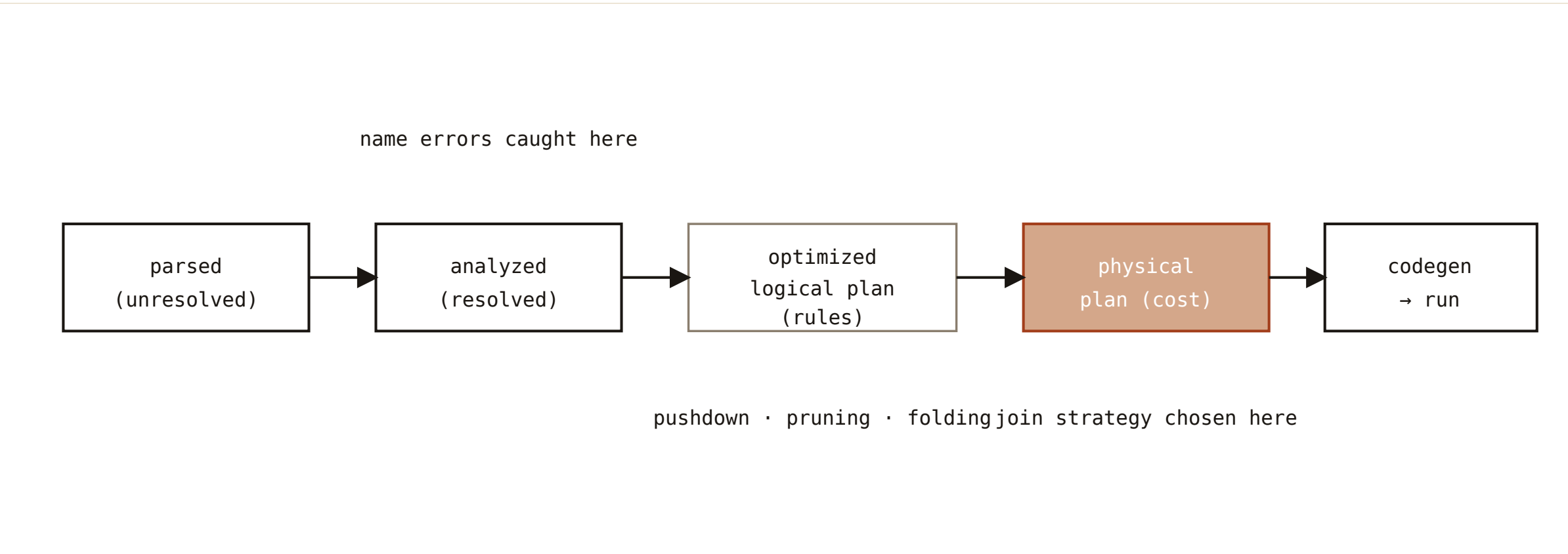

FIG 2.5 *The Catalyst pipeline. Your query is parsed, then resolved against real schemas, then rewritten by optimization rules into a cheaper equivalent, then turned into a concrete physical plan chosen by a cost model, then code-generated and run. The two stages you most often need to inspect are the optimized logical plan (did my filter get pushed down?) and the physical plan (which join did it pick, and where are the shuffles?).*

Reading the tree: explain()

You inspect all of this with `df.explain()`, or `df.explain(True)` to see every stage. The output is a tree, and the first thing to internalize is that **you read it bottom-up**. The leaves at the bottom are the data sources. Execution flows upward toward the root, which is the final result. Here is the physical plan for our running query, lightly cleaned up:

```
== Physical Plan ==
*(2) HashAggregate(keys=[pitch_type], functions=[avg(release_speed)])
+- Exchange hashpartitioning(pitch_type, 200)
   +- *(1) HashAggregate(keys=[pitch_type], functions=[partial_avg(release_speed)])
      +- *(1) Filter (release_speed > 90.0)
         +- *(1) ColumnarToRow
            +- FileScan parquet [pitch_type, release_speed]
                 PushedFilters: [GreaterThan(release_speed, 90.0)],
                 ReadSchema: struct<pitch_type:string, release_speed:double>
```

Read it from the bottom. The `FileScan parquet` is the leaf: Spark reads Parquet. Notice two things on that line. `[pitch_type, release_speed]` means column pruning worked: it is reading two columns, not eighty. `PushedFilters: [GreaterThan(release_speed, 90.0)]` means predicate pushdown worked: the `release_speed > 90` filter is being applied *inside the reader*, so row groups whose statistics rule out matches are skipped before any rows come up. If you expected a filter to push down and you do not see it in `PushedFilters`, that is a finding: something prevented it, and your scan is reading more than it needs.

Moving up: a partial `HashAggregate` computes a per-partition partial average locally. Then comes the line your eye should snap to: `Exchange hashpartitioning(pitch_type, 200)`. **Exchange is the word for a shuffle in a physical plan.** Every Exchange is a wide dependency, a stage boundary, and a place where data crosses the network. Finding the Exchanges in a plan is finding the expensive parts. Here there is exactly one, repartitioning the partial results by `pitch_type` into 200 partitions so that all rows for a given pitch type land together. Above it, the final `HashAggregate` combines the partials into the answer.

The three things to look for in any physical plan

1. **Exchange** nodes: every one is a shuffle. Count them; they are your stage boundaries and your cost.
2. The **FileScan** line: check column pruning (how many columns?) and `PushedFilters` (did your filter make it into the reader?).
3. The **join operator's name**: BroadcastHashJoin, SortMergeJoin, or ShuffleHashJoin (to be discussed), which tells you whether the big side is being shuffled.

The two-phase aggregate, and why it's there

Notice the plan computes the average in two steps: a `partial_avg` below the Exchange and a final `avg` above it. This is not redundancy; it is Spark being clever, and it is worth understanding because it is the model for avoiding shuffle cost in general. The partial aggregate runs *before* the shuffle, on each partition locally, collapsing thousands of `"FF"` rows on a partition into a single partial sum and count. Only those tiny partial results cross the network. Then the final aggregate combines them. Instead of shuffling every raw `(pitch_type, release_speed)` pair, Spark shuffles one small partial per key per partition. The shuffle still happens, but it moves kilobytes instead of gigabytes. This pre-aggregation is the cheapest possible version of a wide operation, and when you see a query that should pre-aggregate but doesn't, you have found a problem.

The marks on the operators

Two notational details so the plan stops looking like noise. The `*` in front of an operator, as in `*(1) HashAggregate`, means that operator was fused into whole-stage code generation; it is part of a single generated loop. The number in parentheses, `(1)` or `(2)`, is the codegen stage id, and it changes exactly at the Exchange. Everything below the Exchange is stage `(1)`; everything above is stage `(2)`. So the stage numbers in the plan line up precisely with the stages we learned about a few classes ago, which line up with the shuffles, which line up with the Exchanges. It is all the same boundary, named four ways.

We can now read a plan and find its shuffles. But we have been treating "the data is in 200 partitions" as a given without asking where that number came from or whether it is right. It turns out the number and arrangement of partitions is the single biggest lever you have over a Spark job's performance, bigger than the cluster size.

Partitioning as *the dominant variable*

In the physical plan last time, the Exchange repartitioned the data into 200 partitions, and we let that number slide by. Now we stop and stare at it, because the number and arrangement of partitions controls more of your job's performance than any other single thing, including how many machines you rent. A student who tunes partitions well on a small cluster will routinely beat a student who tunes nothing on a cluster twice the size. This is the lever.

The partition is the unit of parallelism

Recall that a stage runs as one task per partition. That sentence is the entire reason partitioning matters, so lets follow it to its conclusions. If your data is in 200 partitions, the stage runs as 200 tasks. Those tasks run on your executor cores, as many at once as you have cores. If you have 50 cores and 200 partitions, the cores chew through the tasks in roughly four waves of 50. If you have 50 cores and 8 partitions, then at most 8 cores work and the other 42 sit idle for the entire stage. The partition count is the ceiling on your parallelism. You can rent a hundred cores, but if the data is in eight partitions, you are running an eight-core job.

So partitions interact with the data in two directions at once, and both directions can hurt you. A partition is also the unit of memory: a task loads its partition (and intermediate state) into the memory of one executor core. If partitions are enormous, a single task may not fit in the memory it is given, and Spark spills it to disk or fails outright with an out-of-memory error. So we are squeezed from two sides. Too few partitions starves parallelism and risks blowing memory. Too many partitions drowns the job in per-task overhead.

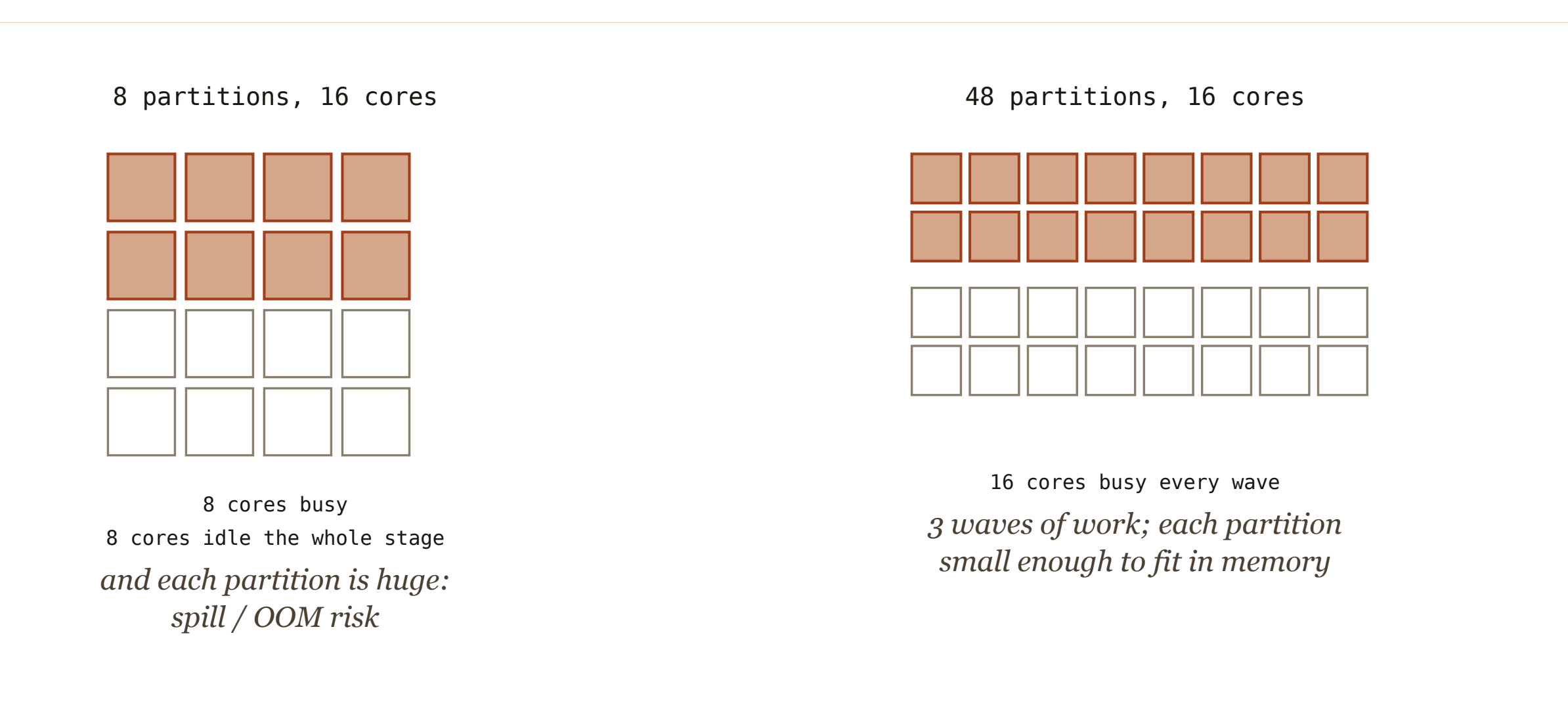
8 partitions, 16 cores
8 cores busy
8 cores idle the whole stage
and each partition is huge:
spill / OOM risk
48 partitions, 16 cores
16 cores busy every wave
3 waves of work; each partition
small enough to fit in memory

FIG 2.6 *Filled boxes are tasks (work); outlined boxes are queued tasks or idle cores. With 8 partitions on 16 cores, half the cluster idles and each partition is large enough to risk spilling. With partitions sized to roughly 2–3× the core count, every core stays busy across a few waves and each partition fits comfortably in a task's memory. Renting more cores does nothing for the left-hand job; repartitioning fixes it for free.*

Where partition counts come from

Three different mechanisms set your partition count, and confusing them is a common source of bafflement.

On **read**, the partition count comes from the data layout. Spark splits input files into partitions of roughly 128 MB by default. Ten 1-GB Parquet files give you about eighty read partitions. Crucially, this is also where the small-files problem from Unit 1 reappears: 100,000 tiny files give you 100,000 read partitions, hence 100,000 tasks, hence the scheduling catastrophe we predicted. The fix is the same as Unit 1's: compact the files, or let Spark coalesce them on read.

After a **shuffle**, the partition count is set by a single configuration value, `spark.sql.shuffle.partitions`, which defaults to **200**. That is where the 200 in our physical plan came from. It is a fixed default that has nothing to do with your data size, which is exactly why it is so often wrong. Shuffle a tiny dataset and you get 200 nearly empty partitions, mostly overhead. Shuffle a huge dataset and you get 200 enormous partitions that spill. The number 200 is right for almost no real workload; it is a placeholder the framework hopes you will reconsider. (Adaptive Query Execution, something we will learn later, was built largely to stop making you reconsider it by hand.)

You can also set the count **explicitly** with `repartition` or `coalesce`, which we turn to now.

repartition versus coalesce

These two look interchangeable and are not, and the difference is, predictably, a shuffle.

`repartition(n)` reshuffles the data into exactly `n` partitions of roughly equal size. It is a full shuffle: every record may move to a new partition, across the network. It can increase or decrease the partition count, and it produces balanced partitions. You pay the shuffle, you get evenness.

`coalesce(n)` only *decreases* the partition count, and it does so without a full shuffle by merging existing partitions on the same executor where possible. It is cheap because it moves little or no data across the network. The catch is that it cannot guarantee balance: if you coalesce 200 uneven partitions down to 10, you may get 10 lopsided partitions, because it merged whatever happened to be adjacent rather than redistributing.

Operation	Shuffle?	Direction	Result balance	Use when
repartition(n)	Yes, full	Up or down	Even	You need balanced partitions, or to increase parallelism
coalesce(n)	No (local merge)	Down only	Possibly uneven	You are shrinking partition count cheaply, e.g. before writing fewer output files

The canonical use of `coalesce` is the end of a job: you have computed a small result across 200 partitions and want to write it a handful of files instead of 200 tiny ones (the small-files problem, on the write side this time). You `coalesce(4)` and write four files, cheaply. The canonical use of `repartition` is the opposite: you are about to shrink fat partitions and need parallelism, so you `repartition(48)` and accept the shuffle to get balanced work for your cores. A frequent mistake is reaching for `coalesce` to *increase* partitions; it silently won't, because it only goes down.

Partitioning by a key, and partitioning on disk

So far we have talked about the *number* of partitions. You can also control the *arrangement*: which rows land in which partition. `repartition("pitch_type")` shuffles so that all rows with the same pitch type land in the same partition. This is exactly what a `groupBy("pitch_type")` needs, so if you partition by the key before aggregating, the aggregation may avoid its own shuffle. Co-partitioning the two sides of a join by the join key, similarly, can let the join skip its shuffle. We will lean on this in the future.

There is a separate, on-disk meaning of partitioning that you met in Unit 1 and must not confuse with the in-memory kind. Writing `df.write.partitionBy("game_date")` lays the data out in directories by date, so that a later query filtered on `game_date` reads only the relevant directories. That is **partition pruning** at the storage layer: a physical directory layout that lets a reader skip whole folders, cousin to the predicate pushdown from Unit 1. In-memory partitioning is about parallelism within one job; on-disk partitioning is about skipping data across many future jobs. Same word, two layers. When someone says "partition," your first question is which layer they mean.

We have now said "shuffle" perhaps thirty times and treated it as a known cost. It is time to stop treating it as a black box. The next lecture opens the shuffle up: what physically happens, why it is the most expensive operation in Spark, and how to make jobs that need fewer of them.

The shuffle, and why it costs *what it costs*

We have been calling the shuffle expensive for weeks and treating it as a black box. Today we open the box. The goal is not trivia; it is that once you know physically what a shuffle does, you can predict its cost from a plan and you know exactly which levers reduce it. Everything in the back half of this unit, joins, skew, and caching, is ultimately about managing shuffles. So let's slow down.

What a shuffle actually is

A shuffle is the operation that redistributes data across the cluster so that records which need to be together end up together. We met the need already: to compute the average for `"FF"`, every `"FF"` row must reach one place. The shuffle is the machinery that gets them there. It happens in two halves with the stage boundary running right between them, and naming the halves is the key to reading shuffle metrics later.

The **map side** is the tail end of the stage *before* the shuffle. Each task, having produced its output records, must decide where each record is going. It computes a destination partition for every record, usually by hashing the key: `hash(pitch_type) mod 200` tells it which of the 200 reduce partitions this row belongs to. The task sorts and groups its records by destination partition and writes them to its *local* disk as a set of **shuffle files**, one logical chunk per destination. Note what just happened: the data hit disk. Even though Spark loves memory, the shuffle write goes to local disk, because the data must survive until the reduce side comes to fetch it, possibly after the writing task is long gone.

The **reduce side** is the start of the stage *after* the shuffle. Each reduce task is responsible for one destination partition, say partition 37. To do its job it must collect every map task's chunk that was destined for partition 37. Those chunks are scattered on the local disks of every node that ran a map task. So the reduce task reaches across the network to every map location and fetches its chunk. This is the all-to-all fetch: if there were `M` map tasks and `R` reduce tasks, there are up to `M × R` fetch connections, because in the worst case every reducer pulls a piece from every mapper.

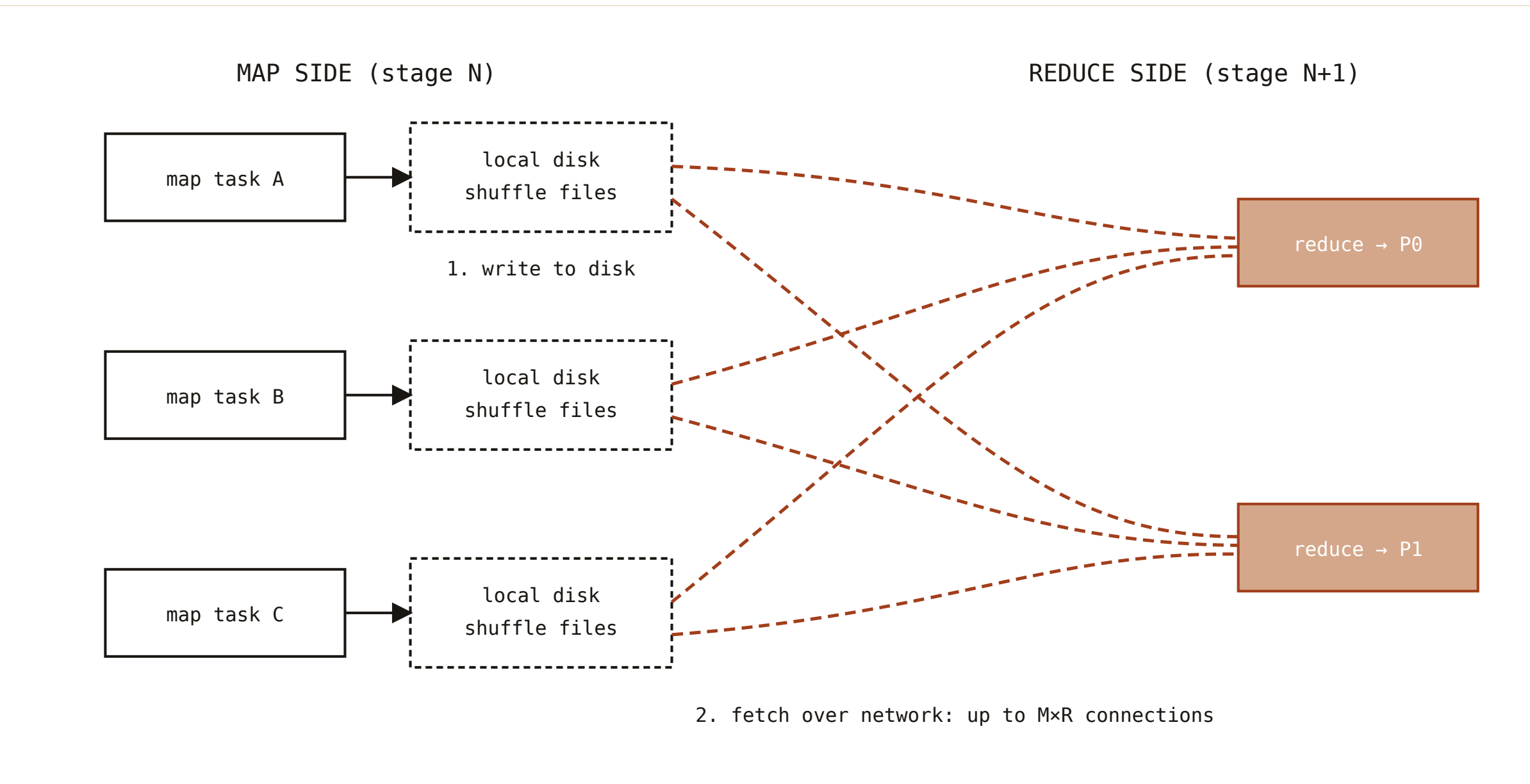

FIG 2.7 *The shuffle in two halves. Map tasks write their partitioned output to local disk as shuffle files (step 1). Reduce tasks then fetch, over the network, the chunk destined for them from every map location (step 2). The data is written to disk once and read back over the network once, and the fetch is all-to-all. This is why a shuffle is the most expensive thing Spark does.*

Why it costs what it costs

Now add up the bills, because each one corresponds to a metric you will read in the Spark UI in the future.

Disk. The map side writes its output to local disk and the reduce side reads it back. That is a full write and a full read of the shuffled data, on top of whatever computation you wanted. Spark is an in-memory engine everywhere except here.

Network. The reduce-side fetch moves the shuffled data across the network, the most expensive transport in Dean's hierarchy from Unit 1. A shuffle of 100 GB moves 100 GB across the wire. There is no avoiding this if the rows genuinely need to be regrouped.

Serialization. Data in memory is in Spark's binary Tungsten format, but to cross the network it must be serialized into bytes and deserialized on the other side. That is real CPU work proportional to the data volume, fallacy seven from Unit 1's list made concrete.

Connections. The `M × R` fetch pattern means many small network connections. With 1,000 map tasks and 1,000 reduce tasks, that is up to a million fetch requests. The per-request overhead, the same kind we saw kill the small-files case, adds up independently of the data volume.

The take-home, the slogan to carry forward: **a shuffle writes the data to disk, reads it back over the network, and re-serializes it, all-to-all.** Compute is fast; the shuffle is slow. When a Spark job is slow, the shuffle is the first suspect, and reading the plan to count Exchanges is how you confirm it.

Spill: when the reduce side doesn't fit

One more cost, and it is the one that turns a slow job into a catastrophically slow one. When a reduce task fetches its data, it tries to hold the working set in memory to process it. If that working set is bigger than the memory the task is given, Spark **spills** it to disk: it writes part of the data out, processes what fits, reads the spilled part back, and merges. Spill shows up explicitly in the UI as "spill (memory)" and "spill (disk)," and it is a direct signal that your partitions are too big for the memory you gave them. The fix is usually more partitions (so each is smaller) or more memory per task. Spill is not an error; the job still finishes. It is just doing two or three times the disk I/O it should, silently, and you only notice because the job is mysteriously slow. Recognizing spill in the UI is a future skill, but the cause is here: partitions that don't fit.

How to need fewer shuffles

You cannot eliminate shuffles; regrouping data is sometimes genuinely required. But many shuffles are avoidable or shrinkable, and someone who knows the moves writes jobs with a fraction of the shuffle cost. Four moves matter.

Pre-aggregate before the shuffle. We already saw this in the two-phase aggregate previously. By collapsing many rows per key into a partial result on the map side, you shrink what crosses the network from gigabytes to kilobytes. Spark does this automatically for built-in aggregations like `avg` and `sum`; the lesson for you is to prefer those over hand-rolled logic that forces every raw row through the shuffle.

Broadcast the small side of a join. If one table is small enough, Spark can send a full copy of it to every executor and do the join locally, with no shuffle of the big table at all. This is the single most valuable shuffle-avoidance trick, and it is so important it gets the whole next lecture.

Co-partition data you will combine repeatedly. If two datasets are already partitioned by the same key, a join between them needs no shuffle, because the matching rows are already co-located. Writing data pre-partitioned (bucketing) trades a one-time shuffle at write for zero shuffles on every future join. Worth it when the same join runs often.

Don't shuffle what you don't have to. Filter early so less data reaches the shuffle. Select only the columns you need so each shuffled row is smaller. Both are things Catalyst tries to do for you, and both are things you can defeat by writing the query in a way that hides your intent, which is one more reason to stay in the structured API.

The most important of these by far is broadcasting, because it converts an expensive wide join into a cheap narrow one. To use it well you have to understand the menu of join strategies Spark chooses from and how it chooses.

Join strategies and *cost-based selection*

We ended last time saying broadcasting converts an expensive join into a cheap one. To use that well you need the whole menu. A join in your code is one word, but Spark has several physically different ways to execute it, and they differ by orders of magnitude in cost. Catalyst picks one for you, usually well, but "usually" is doing work in that sentence, and the times it picks wrong are exactly the times your job is mysteriously slow. So we are going to learn the menu and the chooser, so you can read the choice off a plan and override it when the chooser is wrong.

The running example: we have the big `pitches` table (millions of rows, the fact table) and we want to attach human-readable names. Sometimes we join it against a small `players` table (a few thousand rows mapping `batter` id to name, a dimension table). Sometimes we join `pitches` against another large derived table. These two cases want different strategies, which is the whole point.

Sort-merge join: the default for big ⋈ big

The general-purpose strategy, and Spark's default when nothing smaller applies, is the **sort-merge join**. It works like this. Shuffle both tables by the join key, so all rows with a given key land in the same partition on both sides. Within each partition, sort both sides by the key. Then walk the two sorted streams together with a merge, like merging two sorted lists, emitting matches as the keys line up.

Count the costs against the last lecture and you can see its price immediately: it shuffles *both* tables. Two full shuffles, two sorts. If both tables are large, this is unavoidable and correct; you genuinely have to bring matching keys together and there is no shortcut. But notice what happens when one table is tiny: sort-merge still shuffles the big table across the entire network to co-locate it with a dimension table that would fit in a coat pocket. That is wasteful, and it is the case broadcasting fixes.

Broadcast hash join: the move that skips the shuffle

The **broadcast hash join** applies when one side is small. Instead of shuffling both tables, Spark sends a complete copy of the small table to every executor, where each builds an in-memory hash table from it. Then each executor streams its local partitions of the big table through that hash table, looking up matches. The big table never moves. There is no shuffle of the fact table at all. The only data movement is one broadcast of the small table out to the workers, which for a few-thousand-row dimension table is megabytes, paid once.

This is the difference between shuffling ten seasons of pitch data across the network and not shuffling it. It is the single highest-leverage thing you can do to a join, and recognizing when it applies is a core skill. The catch, and it is a real one, is that the small side has to actually be small, because a full copy lands in the memory of every executor and is first collected to the driver to be broadcast. Broadcast a table that is too big and you blow up the driver's memory, or you push every executor into spill. Broadcasting is a knife: precise and fast, and dangerous if you misjudge the size.

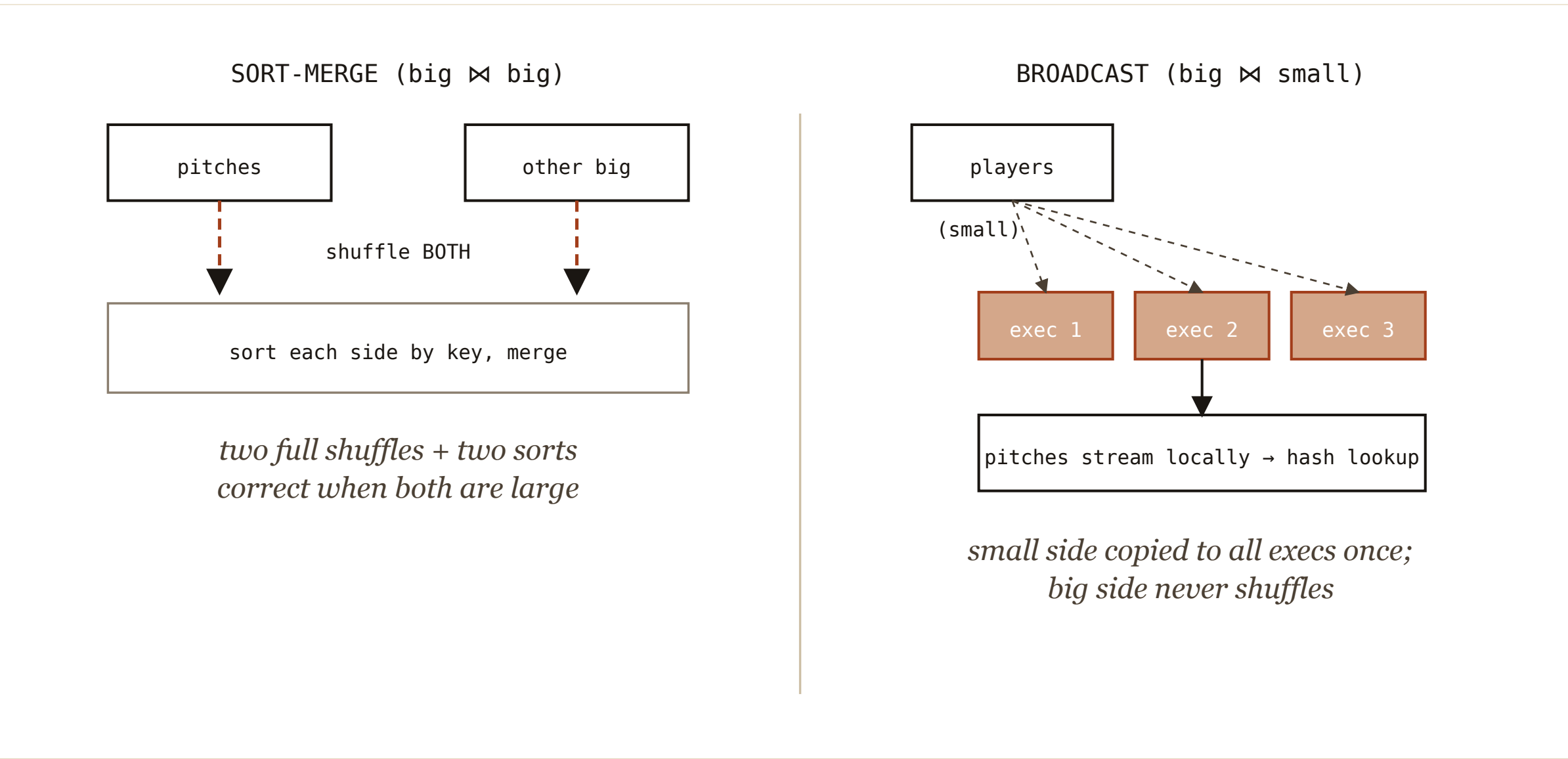
SORT-MERGE (big ⋈ big)
pitches
other big
shuffle BOTH
sort each side by key, merge
two full shuffles + two sorts
correct when both are large
BROADCAST (big ⋈ small)
players
(small)
exec 1
exec 2
exec 3
pitches stream locally → hash lookup
small side copied to all execs once;
big side never shuffles

FIG 2.8 *Two join strategies for the same logical join. Sort-merge shuffles both tables across the network and sorts them; it is the right call when both sides are large. Broadcast sends a copy of the small side to every executor and streams the big side through it locally, shuffling nothing big; it is dramatically cheaper, and it is the right call precisely when one side is small enough to copy everywhere.*

Shuffle hash join: the third option you rarely see

There is a third strategy, the **shuffle hash join**: shuffle both sides by key like sort-merge, but instead of sorting and merging, build a hash table from the smaller of the two shuffled partitions and probe it with the other. It avoids the sort that sort-merge pays. Spark disables it by default (the config `spark.sql.join.preferSortMergeJoin` is true) because sort-merge is more robust to partitions that don't fit in memory, whereas the hash build can blow up if a partition is too large. You will rarely choose it deliberately. Know it exists so the name in a plan doesn't surprise you, and know that if you see it, someone turned off the sort-merge preference on purpose.

How Catalyst chooses

The chooser is mostly about size. The key configuration is `spark.sql.autoBroadcastJoinThreshold`, which defaults to **10 MB**. If Spark's statistics say one side of a join is smaller than that threshold, it picks a broadcast hash join automatically. Otherwise it falls back to sort-merge. So the entire automatic decision hinges on whether Spark *believes* a table is under 10 MB, which raises the obvious failure mode.

Why the chooser goes wrong

Catalyst's size estimate is only as good as its statistics. If your dimension table is genuinely 4 MB but Spark has no statistics and estimates it at 4 GB (because it is the output of an opaque upstream operation it can't size), Spark will not broadcast it, and you will pay a full sort-merge shuffle of the fact table for no reason. The fix is either to compute table statistics so Spark estimates correctly, or to tell Spark directly with a **broadcast hint**.

The broadcast hint is your override. `df.join(broadcast(players), "batter")` tells Catalyst "I promise this side is small, broadcast it," skipping the size guess entirely. This is one of the few places where you routinely know something the optimizer doesn't (the true size of your dimension table) and should say so. The mirror-image danger applies: if you hint a broadcast on a table that is actually large, you have ordered Spark to collect a large table to the driver and copy it everywhere, and you will get a driver out-of-memory error that points nowhere obvious. Hint broadcasts only when you are sure the side is small.

Reading the choice off the plan

This all becomes concrete the moment you call `.explain()` on a join. The operator name tells you exactly which strategy was chosen:

Plan shows	Strategy	What it means for cost
BroadcastHashJoin	Broadcast	One side broadcast; big side not shuffled. Look for the `BroadcastExchange` below it.
SortMergeJoin	Sort-merge	Both sides shuffled and sorted. Look for an `Exchange` under each side.
ShuffleHashJoin	Shuffle hash	Both shuffled, no sort, hash build. Rare; someone changed the default.

So the workflow when a join is slow is mechanical, and it is the workflow you will practice in lab. Call `explain`. Find the join operator. If it says `SortMergeJoin` and you believe one side is small, ask why it didn't broadcast: is the size estimate wrong? Add statistics or a broadcast hint and re-check the plan. If it says `BroadcastHashJoin` on a side you thought was large, you may be about to OOM the driver. The plan is the diagnosis; the hint is the treatment.

Even a perfectly chosen join, though, can run slowly for a reason that has nothing to do with strategy. If almost all the rows share one join key, then one reduce task gets almost all the work while the rest finish instantly, and the whole stage waits on that one straggler. That is skew, and it is so common and so confusing in the UI that it gets its own lecture, along with the machinery Spark added to fight it automatically.

Skew, salting, and *Adaptive Query Execution*

We ended last time on a job that picks the right join and is still slow because of one straggler task. That is skew, and it deserves a full lecture for a specific reason: it is the hardest performance problem to diagnose, because the plan looks perfect and most of the cluster looks idle, yet the job crawls. Once you have seen the shape, you will recognize it everywhere. Lets build the shape.

What skew is

A shuffle partitions data by key, usually by hashing the key. That works beautifully when keys are evenly distributed. But real data is rarely even. Suppose we group Statcast by `batter`, and one batter has ten times as many plate appearances in our slice as a typical one, or worse, suppose a data-quality problem has left a million rows with a null `batter` that all hash to the same partition. After the shuffle, one reduce partition is enormous and the rest are small.

Now recall the iron law from earlier in this unit: a stage finishes when its *last* task finishes. The 199 small partitions get processed in seconds. The one giant partition takes minutes. The stage is not done until that one task is done, so the whole stage takes minutes, and for almost all of those minutes 199 of your 200 cores are sitting idle, having finished their tiny partitions, waiting on the straggler. You rented a big cluster and you are running a one-core job at the end. This is **data skew**, and the giant task is the long pole.

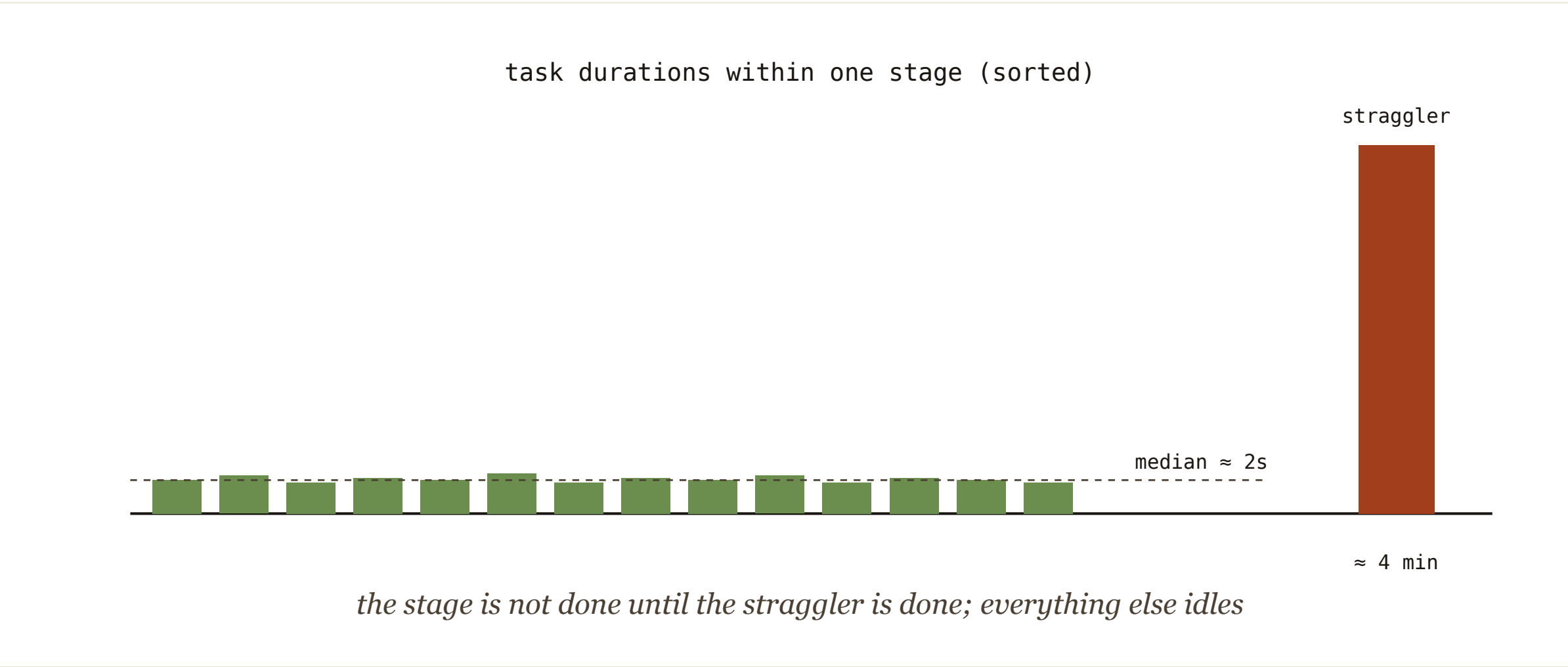

FIG 2.9 *The signature of skew, exactly as the Spark UI's task-duration summary shows it. Most tasks finish in about two seconds (the median); one task runs for minutes because its partition holds a hot key. The gap between the median task and the max task is the tell. When you see median and max an order of magnitude apart in a stage, you have skew, and no amount of extra hardware helps the straggler.*

Detecting it

The detection is the figure above, read off the Spark UI. Open the slow stage. Look at the summary of task durations: minimum, 25th percentile, median, 75th, max. In a healthy stage these are all close. In a skewed stage the max is wildly larger than the median, and the same blowup shows in the "shuffle read size" column, because the straggler task read a far larger partition than its peers. That divergence between median and max is the diagnosis. It is, in practice, one of the most common reasons a real Spark job underperforms, and it is invisible in the physical plan, which is exactly why the UI skill matters.

The manual fix: salting

The classic manual cure is **salting**, and the idea is to artificially break the hot key into several keys so its rows spread across several partitions. You append a random suffix, the salt, to the join or group key. The one giant `batter = X` key becomes `X-0, X-1, X-2, ... X-9`, ten keys that hash to ten different partitions, so the work that crushed one task now spreads across ten. On a join you have to salt both sides compatibly so matches still find each other (replicating the small side across all salt values), and after a salted aggregation you do a second, cheap aggregation to combine the per-salt partials back into the per-key answer. It works, and for years it was the only answer, but it is fiddly, intrusive to your query, and it requires you to know *which* key is hot ahead of time.

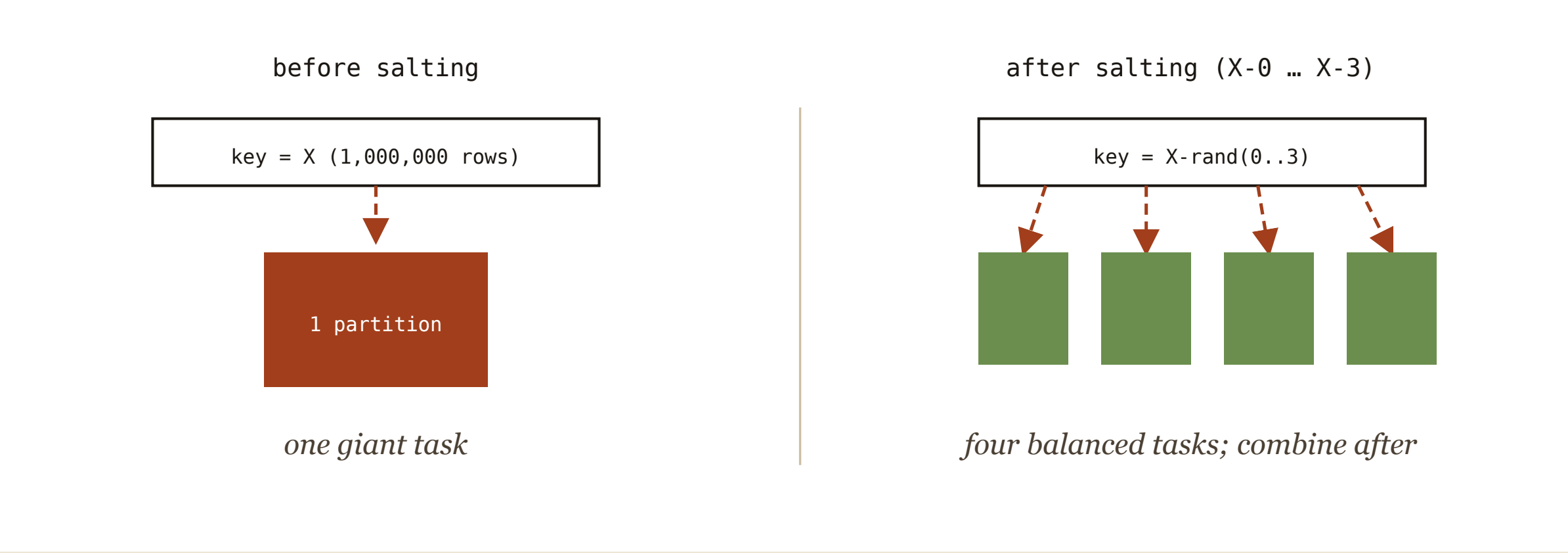

FIG 2.10 *Salting spreads a hot key across several partitions by appending a random suffix, turning one overloaded task into several balanced ones. The cost is a more complicated query (you must replicate the join's small side across salt values, and re-aggregate the salted partials), and the requirement that you already know which key is hot.*

Adaptive Query Execution: the optimizer that watches itself run

The fact that you had to know the hot key in advance points at the deeper problem with everything we have done so far: Catalyst plans the whole job *before* running it, using statistics that are often missing or wrong. It guesses 200 shuffle partitions because it can't see the real data sizes. It guesses join strategies from estimates. **Adaptive Query Execution**, AQE, fixes this by letting the optimizer revise the plan at each shuffle boundary using the *actual* sizes it just measured. It has been on by default since Spark 3.2, so it is already helping you, and understanding what it does explains a lot of behavior that would otherwise look like magic. It does three things.

It coalesces shuffle partitions dynamically. Remember the dumb 200 default. After a shuffle, AQE looks at how big the partitions actually came out and merges the small ones together to hit a sensible target size. If your shuffle produced 200 tiny partitions, AQE quietly combines them into, say, 12 reasonable ones, so you stop paying for 200 nearly empty tasks. This alone removes most of the reason people used to tune `spark.sql.shuffle.partitions` by hand.

It handles skew joins automatically. AQE detects, from the real post-shuffle sizes, that one partition is far larger than the median (by default, more than five times the median and over 256 MB), and it splits that oversized partition into several sub-partitions, doing automatically what salting did by hand. You often no longer need to salt at all, which is a large quality-of-life improvement and the reason salting is now a fallback rather than a first resort.

It switches join strategies at runtime. If Catalyst planned a sort-merge join because it estimated both sides were large, but the actual shuffle reveals that one side came out small enough to broadcast, AQE switches the plan to a broadcast join on the spot. The estimate was wrong; the measurement corrected it. This is the cleanest illustration of AQE's whole philosophy: plan with guesses, then fix the plan with facts.

What AQE changes about how you work

AQE does not make skew and partition tuning disappear from your job; it makes them disappear from your *code* in the common cases. You still need to recognize skew in the UI, because AQE has thresholds and does not catch everything, and because when it does act, the plan you read with `explain` and the plan that actually ran will differ. To see the plan AQE actually executed, you read the final plan in the SQL tab of the UI, not the static `explain` output. That is one more reason the UI, not the plan printout, is the ground truth.

We have now built the whole performance toolkit: partitions, shuffles, joins, skew, and the adaptive machinery that tunes them. There is one resource we have leaned on constantly and never managed deliberately: memory. We keep saying Spark holds intermediate results in memory, but we have not decided what to keep, where, or in what form. Controlling that deliberately is caching.

Caching and *persistence*

We have been saying since the beginning of this unit that Spark keeps things in memory, and that is the difference from MapReduce. But we have not decided what to keep. By default Spark keeps nothing across actions: each action recomputes the whole lineage from the source. That is the right default, and the cases where you should override it are specific. Caching the wrong thing is one of the most common ways to make a Spark job slower while believing they made it faster, so this lecture is as much about when *not* to cache as when to.

The default is recompute, and usually that's correct

Recall the recipe model that a DataFrame is a lineage, not stored data. So watch what happens when you run two actions on the same DataFrame:

```
fast = spark.read.parquet("s3a://sd411/statcast/").filter("release_speed > 90")
fast.count()                         # job 1: reads + filters from scratch
fast.groupBy("pitch_type").count().show()   # job 2: reads + filters AGAIN
```

The second action does not reuse the first action's work. It re-reads the Parquet and re-applies the filter from the source, because the lineage says "this is the filtered read" and Spark dutifully recomputes it. For two actions over a cheap pipeline, that double read is fine and probably not worth caching. But if `fast` were the result of an expensive chain of joins and aggregations, recomputing it for every action would be wasteful, and that waste is exactly what caching removes.

Caching materializes the recipe once

Calling `fast.cache()` (or the more explicit `fast.persist()`) tells Spark: the first time you compute this, keep the result, and serve every later action from the kept copy instead of recomputing the lineage. After caching, job 1 computes and stores `fast`; job 2 reads it straight from memory, skipping the Parquet read and the filter entirely.

Two details trip people up. First, `cache()` is itself *lazy*, like every transformation. It marks the DataFrame to be cached but stores nothing until the next action forces the computation. So the first action after a `cache()` is no faster; it does the work and fills the cache. Only later actions benefit. Students who call `cache()` and time the very next action conclude caching did nothing, because they measured the fill, not the reuse.

Second, caching truncates lineage for recovery purposes. Once a partition is cached, a downstream failure can rebuild from the cache rather than replaying the whole lineage from source, which is the in-memory version of the checkpointing we previously flagged. This is a real benefit for long iterative jobs independent of the recompute savings.

When caching actually pays, and when it just costs

Caching is worth it when a DataFrame is both *expensive to compute* and *used more than once*. Both conditions have to hold. The textbook case is an iterative algorithm: gradient descent or the K-means you already know or the PageRank you'll soon meet reads the same prepared dataset on every iteration. Cache the prepared dataset once, and all the iterations read from memory instead of recomputing. This is precisely the workload MapReduce was bad at, and caching is how Spark is good at it.

Caching is a mistake when either condition fails. If you cache a DataFrame and then use it exactly once, you have paid the cost of filling the cache for no reuse benefit, and worse, you have consumed memory that other work needed. If you cache something cheap to recompute, the recompute would have been faster than the memory pressure you created. And caching is actively harmful when the cached data is large enough to evict *other* things Spark needed in memory, including the shuffle working sets from the last few lectures, pushing them to spill. Cache is a finite budget. Spend it on the expensive, reused thing, and spend nothing on anything else.

The cache discipline

Cache only what is expensive *and* reused. Remember that `cache()` is lazy, so the win shows up on the second action, not the first. And call `unpersist()` when you are done with a cached DataFrame, to hand the memory back. Spark will evict caches automatically under pressure, but explicit `unpersist` is how you keep a long job from quietly starving its own later stages.

Storage levels: where and in what form

`persist()` takes a storage level that answers three questions: memory or disk or both, serialized or not, and replicated or not. `cache()` is just `persist()` with the default level. The menu:

Storage level	Where	Form	Use when
MEMORY_ONLY	RAM only; partitions that don't fit are recomputed on demand	Deserialized objects	Fastest reuse, data fits comfortably (the RDD default)
MEMORY_AND_DISK	RAM, spilling the overflow to local disk	Deserialized in RAM	Safe general default; the DataFrame `cache()` default
MEMORY_ONLY_SER	RAM only	Serialized (compact bytes)	Memory-tight; trade CPU for space
MEMORY_AND_DISK_SER	RAM + disk overflow	Serialized	Memory-tight and can't afford recompute on overflow
DISK_ONLY	Local disk	Serialized	Recompute is very expensive but data is too big for RAM
..._2 (e.g. MEMORY_ONLY_2)	As above, replicated on two nodes	As above	Recompute is so costly you want a redundant cached copy

The two axes worth reasoning about are the serialized-versus-not and the memory-versus-disk tradeoffs, and both are the same tradeoff you have seen all throughout this unit: space against time. Serialized caching stores compact bytes, so more fits in RAM, but every read must deserialize, costing CPU. Disk-backed levels survive eviction without recompute, but reads pay disk latency. The default `MEMORY_AND_DISK` for DataFrames is a sensible compromise: fast deserialized access in RAM, with overflow spilling to disk rather than being lost. You should reach for the other levels only when you have measured a specific pressure: out of memory, push toward serialized or disk; recompute too expensive to risk, push toward disk-backed or replicated. Otherwise the default is the right answer, which is the recurring theme of this course: change the default only when a measurement tells you to.

Cache versus the storage layout from Unit 1

One clarification, because the words overlap. Caching is a within-job, in-memory decision about reusing a computed result during one Spark application. It vanishes when the application ends. The on-disk Parquet layout, partitioning, and compaction from Unit 1 are durable, cross-job decisions about how bytes live on storage between applications. They solve different problems. A well-laid-out Parquet table makes every job's *read* cheap; caching makes *reuse within one job* cheap. You will often want both, and confusing them leads to people trying to "cache" data permanently or "partition" data for a single analysis. Keep the layers straight: storage layout is Unit 1, caching is here.

We now have every concept in the unit: the execution model, the plan, partitions, shuffles, joins, skew, AQE, and caching. What we have not done is sit in front of a slow job and use all of it at once.

Spark UI forensics, and *resource sizing*

Every class in this unit has handed you one instrument: the recipe model, the physical plan, partition counts, shuffle accounting, join strategy, the skew signature, AQE's adjustments, the cache decision. Each was introduced with a way to see it. Today is where we sit in front of a job that is too slow and use all of them at once, because in practice no one hands you a labeled symptom. You get a job that took eleven minutes when you expected two, and a web page full of numbers, and the skill is knowing which number to read first. The second half answers the question that has been hovering since the beginning of this unit and that you cannot avoid once you run on a real cluster: how big a cluster should you ask for in the first place?

The UI is the plan, made of measurements

The Spark UI is a web page the driver serves while an application runs, on port 4040 by default, and it is organized into tabs that map exactly onto the model we built. The Jobs tab lists each action you triggered. Click a job and you see its stages, which are the shuffle-bounded segments. Click a stage and you see its tasks, one per partition, with a duration for each. The SQL tab shows the query as a diagram of operators annotated with the rows and bytes that actually flowed through them. The Executors tab shows the machines: how much memory each holds, how much time each spent in garbage collection, how much data each read and wrote. The Storage tab shows what is currently cached and how much of it fits in memory. Nothing here is new physics. It is the same execution model from the rest of the unit, with real numbers filled into the boxes that `explain()` left empty.

The single most important consequence of AQE, from a forensic standpoint, is this: the SQL tab shows the plan that *actually ran*, after AQE's runtime rewrites, while the static `explain()` shows the plan as Catalyst guessed it *before* running. When the two disagree, the SQL tab is the truth. A join you wrote expecting a sort-merge may appear in the SQL tab as a broadcast, because AQE measured the build side at runtime and switched. So when you are diagnosing a real job, the SQL tab is ground truth and the static plan is intent.

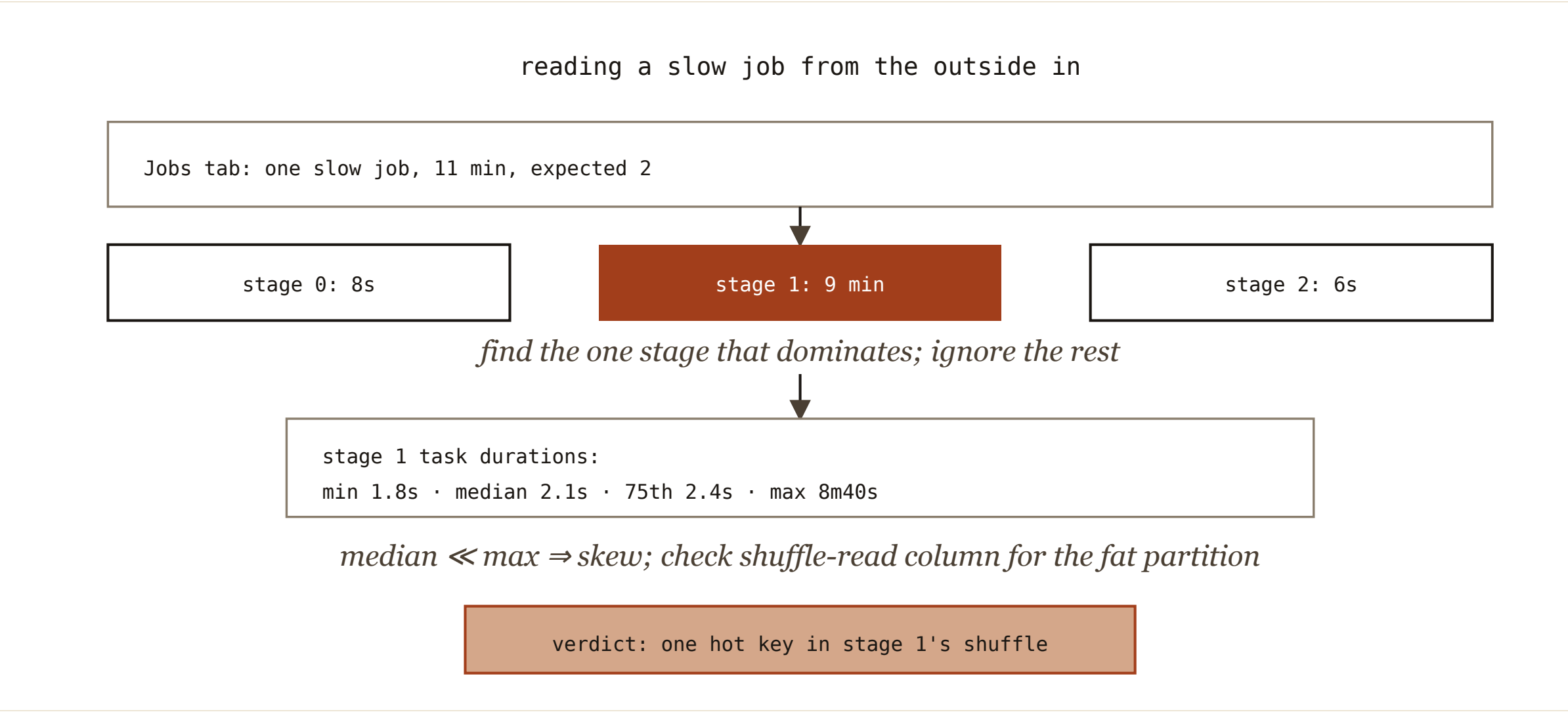
reading a slow job from the outside in
Jobs tab: one slow job, 11 min, expected 2
stage 0: 8s
stage 1: 9 min
stage 2: 6s
find the one stage that dominates; ignore the rest
stage 1 task durations:
min 1.8s · median 2.1s · 75th 2.4s · max 8m40s
median ≪ max ⇒ skew; check shuffle-read column for the fat partition
verdict: one hot key in stage 1's shuffle

FIG 2.11 *The forensic order is outside in: a job decomposes into stages, and one stage almost always dominates the wall-clock time. Open that stage, read the task-duration summary, and the shape of the numbers names the problem. A median far below the max is the skew signature. Uniformly slow tasks with large shuffle-read bytes point at an unavoidably large shuffle. Uniformly slow tasks with high spill point at too few partitions for the memory available. You never read every number; you follow the dominant stage down to its tasks.*

A diagnostic order that works

The method in the figure is worth stating as a habit, because without one, you tend to stare at the Executors tab hoping a cause jumps out. Spoiler Alert: It does not. Start at the Jobs tab and find the job that took the time. Open it and find the stage that dominates the job; there is almost always one stage that is most of the wall clock, and the others are rounding error. That single decision, which stage, throws away ninety percent of the page. Now open the dominant stage and read the task-duration summary, the same five numbers: min, 25th, median, 75th, max. Their shape is the diagnosis.

If median and max are far apart, you have skew, and you confirm it by sorting tasks on shuffle-read size to see the one fat partition; the fix is AQE's skew handling or manual salting. If every task is slow and the shuffle-read bytes are large and even, the shuffle itself is genuinely big, and the fix is upstream: filter earlier, or broadcast the small side so there is no shuffle at all. If every task is slow and the spill metrics are nonzero, each partition is too big for the memory each task was given, so the fix is more partitions, which makes each one smaller. If the stage is fast but there are too few tasks to fill your cores, you are under-partitioned and wasting the cluster, the underutilization picture. Each symptom in the task summary points back at exactly one class from this unit. That is the payoff of having built the model in order: the UI is not a new subject, it is the index to everything you already understand.

Spill is the metric students miss

Two numbers in the stage detail are labeled "spill (memory)" and "spill (disk)." Spill means a task could not fit its working data in memory and wrote it to local disk mid-computation, then read it back. It is not the same as the deliberate disk caching; it is involuntary, and it is pure overhead. Nonzero spill in a stage is a direct instruction: give that stage more partitions so each task handles less, or give the executors more memory. Spill is the cluster telling you, in plain numbers, that your partition sizing was too coarse for the memory you have.

Sizing the cluster: the three numbers you actually choose

Now the question we have deferred. When you launch a Spark application you are not handed an infinite machine; you request a shape, and the shape is three numbers: how many executors, how many CPU cores each executor gets, and how much memory each executor gets. An executor is a single JVM process running on a worker node, holding some cores and some memory, and it is where tasks actually run. Cores set how many tasks an executor runs at once, since one task occupies one core. Memory is shared across those concurrent tasks plus any cached data. Getting these three numbers roughly right is most of the difference between a job that uses the cluster you paid for and one that leaves it idle.

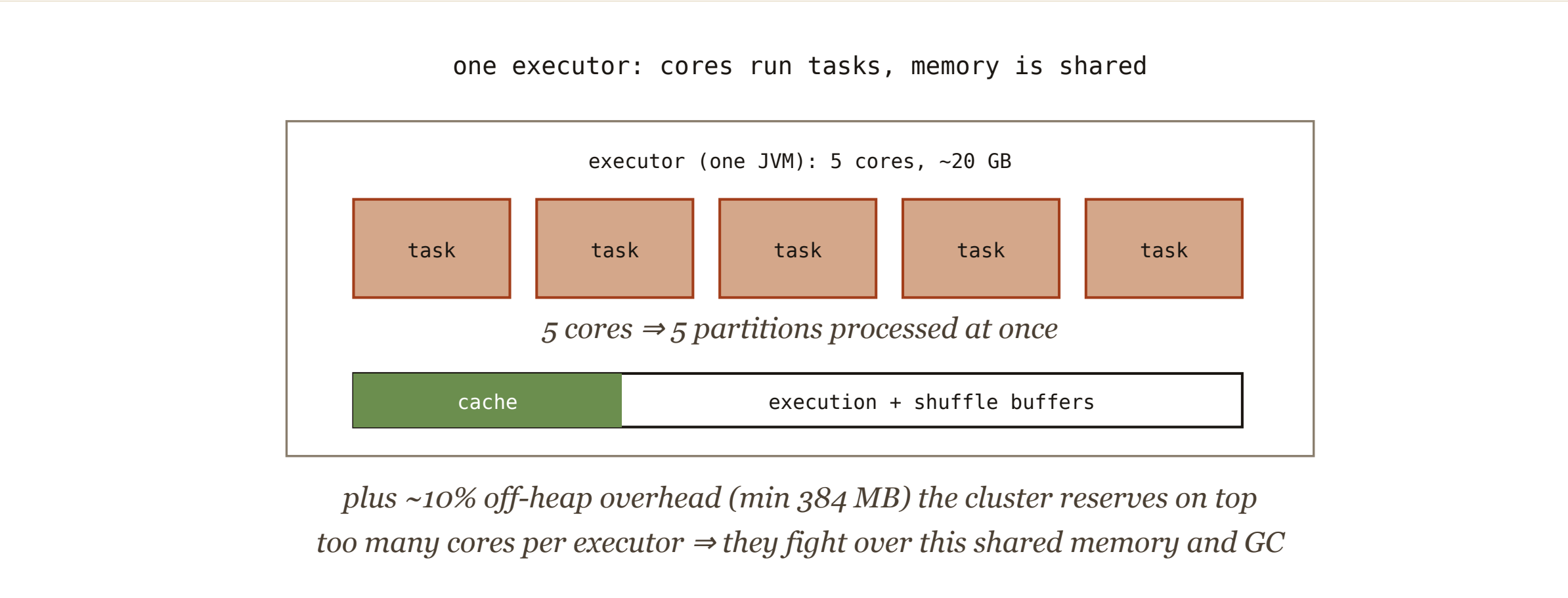
one executor: cores run tasks, memory is shared
executor (one JVM): 5 cores, ~20 GB
task
task
task
task
task
5 cores ⇒ 5 partitions processed at once
cache
execution + shuffle buffers
plus ~10% off-heap overhead (min 384 MB) the cluster reserves on top
too many cores per executor ⇒ they fight over this shared memory and GC

FIG 2.12 *An executor is one JVM with a fixed number of cores and a fixed memory pool. The cores set its task parallelism, one task per core, so five cores means five partitions processed at once. The memory pool is shared by all those concurrent tasks plus any cached partitions, which is why packing too many cores into one executor backfires: the tasks contend for the same heap and the garbage collector stalls them all. The cluster also reserves roughly ten percent on top of your requested heap, with a 384 MB floor, for off-heap overhead, so a request for 18 GB of usable heap actually consumes about 20 GB.*

The defaults the community has converged on are worth memorizing as starting points, not laws. Give each executor about five cores. Fewer wastes the per-executor memory overhead across too little work; many more, and the concurrent tasks contend for the executor's shared heap and the garbage collector starts stalling everything, so throughput per core falls even though the count looks impressive. Five is the knee of that curve. From there, the number of executors is set by how many total cores you want and can afford: total cores divided by five. Memory per executor is whatever the node can spare after leaving room for the operating system, remembering that the cluster manager adds roughly ten percent overhead, with a 384 MB minimum, on top of the heap you request. And the partition count, the variable that turned out to govern everything, should be set so there are about two to three partitions per total core, which gives every core a few units of work to chew through and lets AQE coalesce away the excess when the data turns out small.

You choose	It controls	Reasonable starting point
Cores per executor	Tasks running at once per JVM; heap contention	≈ 5
Number of executors	Total parallelism; total cost	(total cores you want) ÷ 5
Memory per executor	Cache space + execution buffers per JVM	What the node spares, less ~10% overhead
Shuffle partitions	Task granularity through shuffles	≈ 2–3 × total cores (AQE trims the excess)

None of these is a number you compute once and trust forever. They are a starting shape that you then correct using the forensic skill from the first half of this lecture. You launch with the defaults, you read the UI, and you adjust the one number the symptom names: spill says more partitions or more memory, idle cores say more partitions or fewer executors, GC stalls say fewer cores per executor. Sizing and forensics are the same loop, which is the honest summary of operating Spark: there is no formula that replaces measuring the job you actually have.

That loop, measure then adjust one variable, is the last and most general instrument in the unit, and it is the one that outlasts Spark itself. The next unit changes the shape of the problem entirely: the data stops arriving as a finished file we can plan against and starts arriving as an unbounded stream that never ends, so there is no whole dataset to count, no final answer to compute, and the very idea of a job that starts and finishes has to be rebuilt.